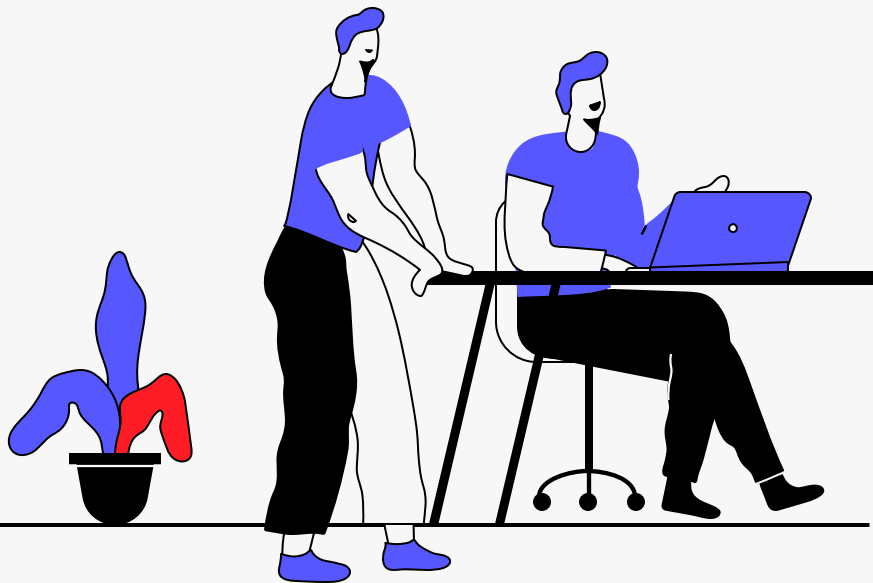


Debuggeando en BlueJ



Introducción a la Programación
Orientada a Objetos





Utilizando el debugger

- Todos los ambientes de desarrollo incorporan herramientas de depuración de código. **BlueJ** no es la excepción.
- Un debugger o depurador de código es una herramienta que nos va a permitir ejecutar nuestro código en un modo especial para:
 - seguir la ejecución del programa de manera detallada y observar el flujo de control de nuestro programa
 - observar el contenido de variables, parámetros y atributos
 - examinar la pila de llamadas del sistema (call stack) para observar el contexto de ejecución
 - observar los objetos en ejecución y sus referencias





Utilizando el debugger

- Un debugger o depurador de código es una herramienta que nos va a permitir ejecutar nuestro código en un modo especial para:
 - controlar la ejecución deteniéndola en puntos específicos (breakpoints)
 - verificar el contenido de memoria
 - aplicar técnicas de debugging avanzadas que no veremos en esta introducción
- **Y todo lo anterior SIN ENSUCIAR/MODIFICAR nuestro código!**





Pasos para debuggear

1. Iniciar el modo de depuración (debugger) del IDE.
 - El código debe compilar correctamente
2. Establecer correctamente el contexto de depuración (i.e. llegar al lugar del código donde quiero depurar y en el contexto de ejecución adecuado)
 - Establecer los breakpoints adecuados
 - Verificar que la ejecución se da en un contexto de invocación de métodos relevante para el objetivo de depuración (ver el call stack)
3. Examinar el flujo de ejecución (step over/step into) y el contenido de memoria (variables). Verificar si son los esperados (ok) o si difieren (bugs).
4. Una vez hallado el problema, cerrar la sesión de debug, corregir el código y comprobar que el problema está resuelto.



- • •
- • •
- • •
- • •
- • •

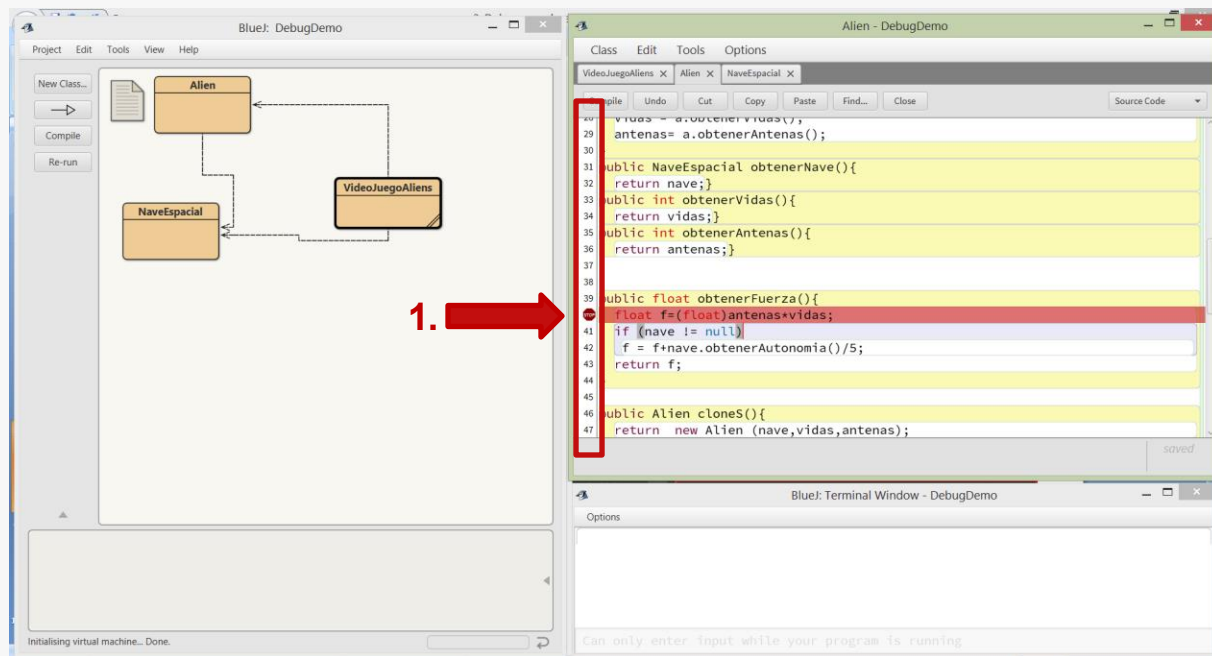
Iniciando el debugger en BlueJ

1. Inserta al menos un Breakpoint (punto de ruptura) en alguna línea de código ejecutable que sea alcanzada por la ejecución del código. Hacer click en el margen indicado, en la línea seleccionada, para que aparezca el punto rojo.

Son líneas ejecutables de código aptas para insertar breakpoints:

- Asignaciones
- Instrucciones en las que se evalúen expresiones
- Instrucciones en las que se envíen mensajes a objetos

No se insertan breakpoints en líneas en blanco, comentarios ni declaraciones.



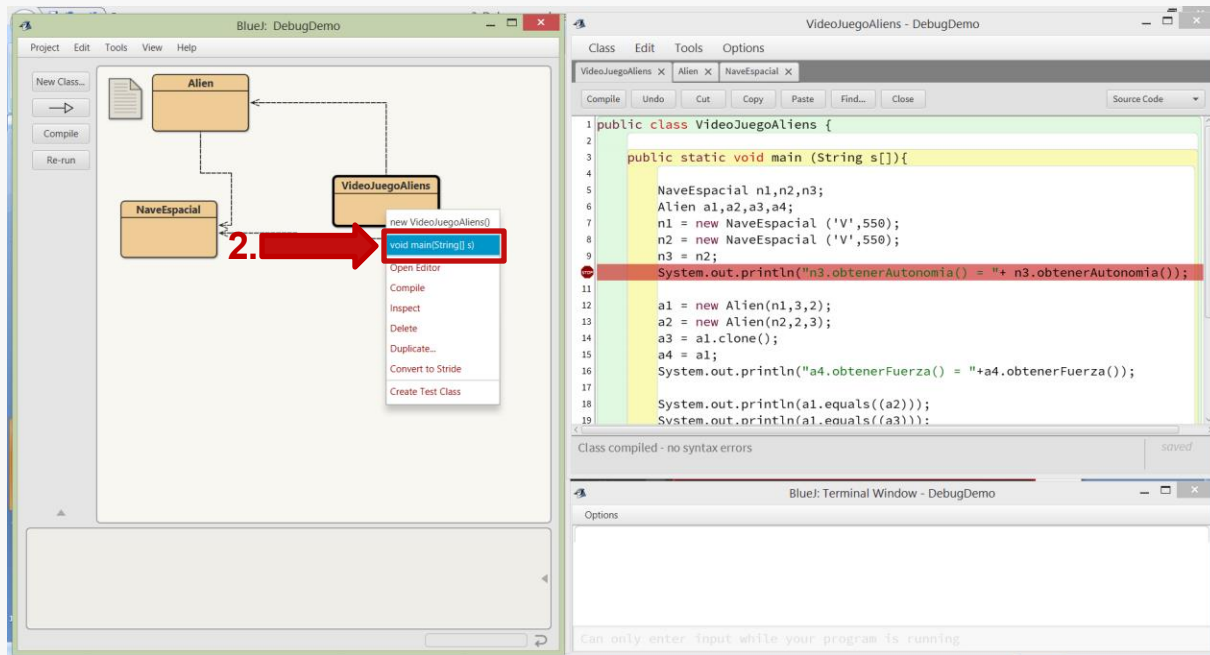
- • •
- • •
- • •
- • •

Iniciando el debugger en BlueJ

2. Dirígete a la clase con el método main, hacé click derecho y seleccioná el método main para iniciar la ejecución. Al haber breakpoints establecidos se iniciará la ejecución en modo debug.

Esto inicia la ejecución del programa en modo debug. La misma se detendrá en el primer breakpoint que se alcance.

Si no se alcanza ningún breakpoint, la ejecución progresará hasta su finalización (como en el caso de una ejecución normal)





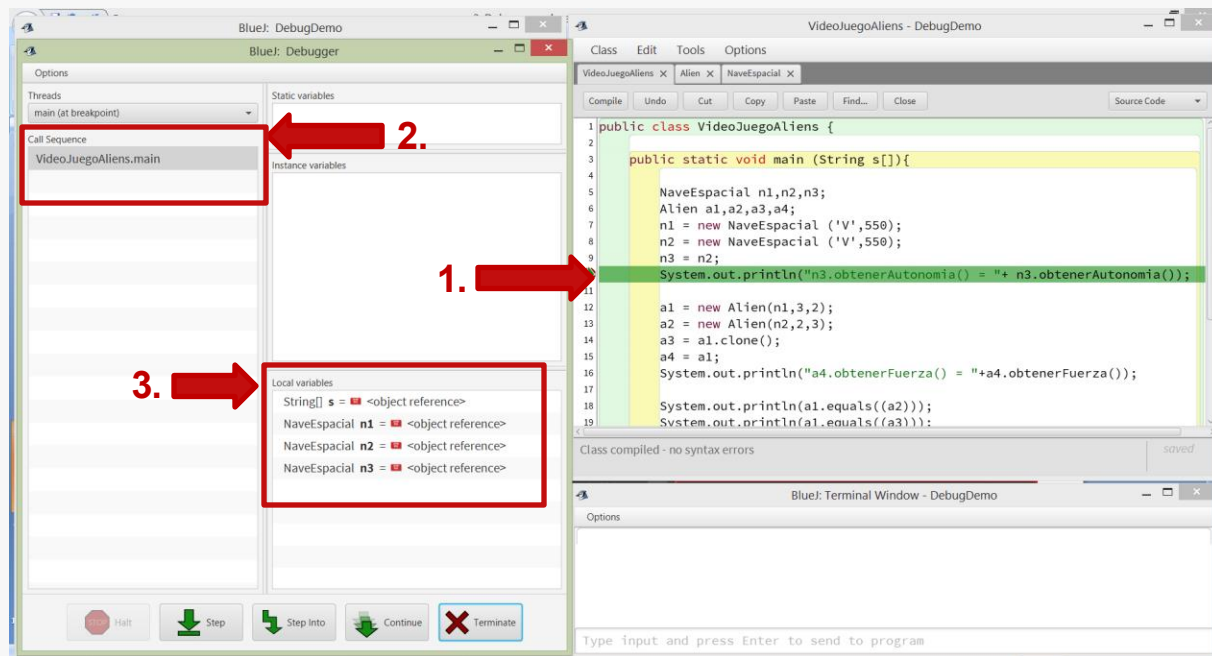
Iniciando el debugger en BlueJ

3. La vista de Debugger de BlueJ muestra información adicional de depuración en la sesión de depuración actual.

1. Se indica/resalta la línea de ejecución actual

2. Call Sequence: la pila de invocaciones de los servicios de Java

3. Variables: incluye el contenido de memoria (variables, parámetros, atributos)



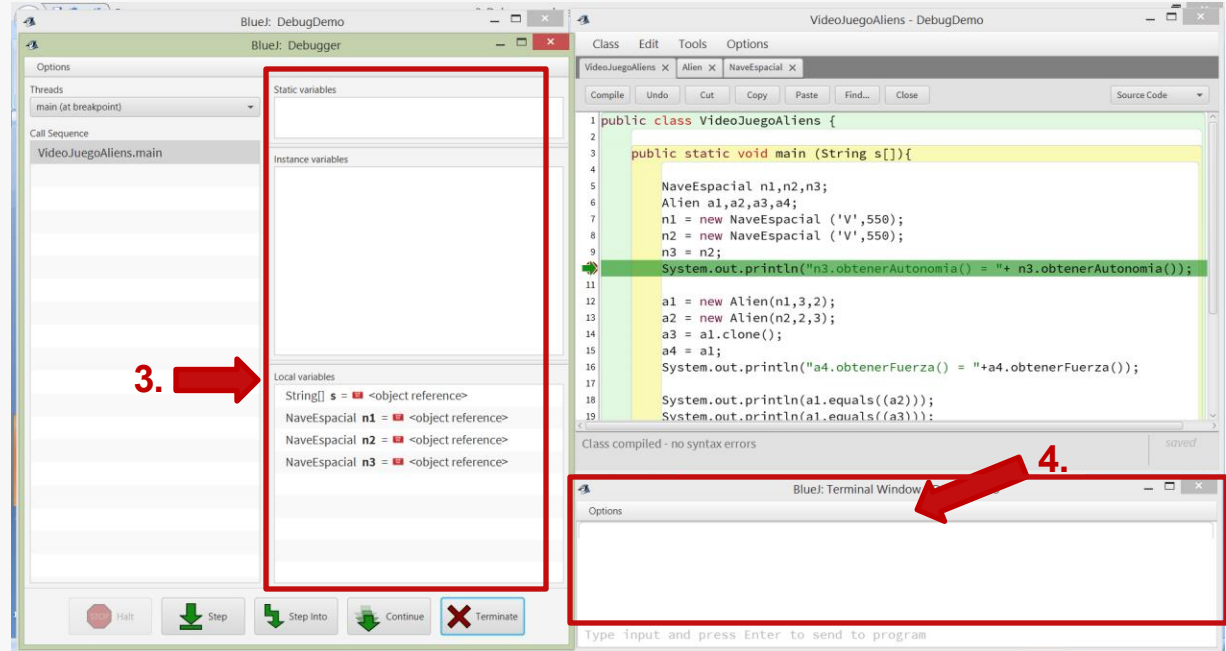
Iniciando el debugger en BlueJ

3. La vista de Debugger de BlueJ muestra información adicional de depuración en la sesión de depuración actual.

3. Variables: incluye el contenido de memoria:

- Atributos de clase (static variables)
- Atributos de instancia (instance variables)
- Variables locales

4. La consola (Terminal), al igual que en ejecución normal, permite hacer Entrada/Salida con el usuario.

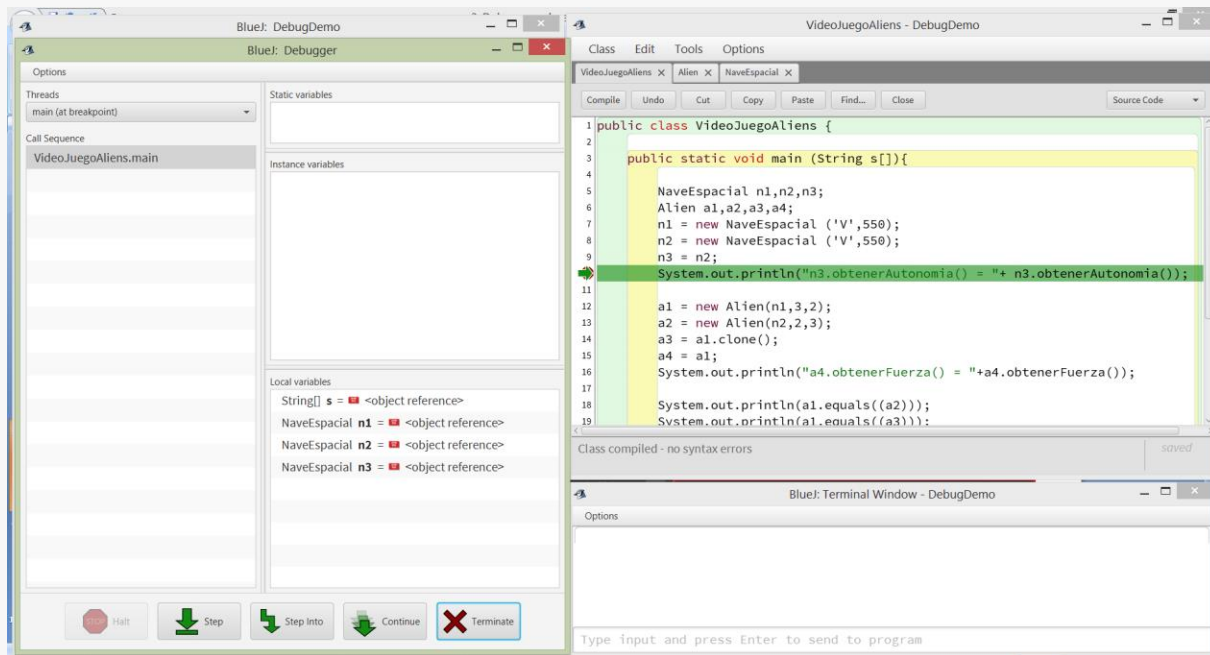


- • •
- • •
- • •
- • •
- • •
- • •

Iniciando el debugger en BlueJ

3. La vista de Debugger de BlueJ muestra información adicional de depuración en la sesión de depuración actual.

En el ejemplo la ejecución se encuentra detenida en la línea 10. Ya se ejecutaron las líneas 7, 8 y 9. Esto se puede ver en **Local Variables**, el ambiente de referenciamiento local al método actual (main), e incluye sus parámetros formales (s) y sus variables locales (n1, n2 y n3 son las únicas inicializadas hasta el momento).

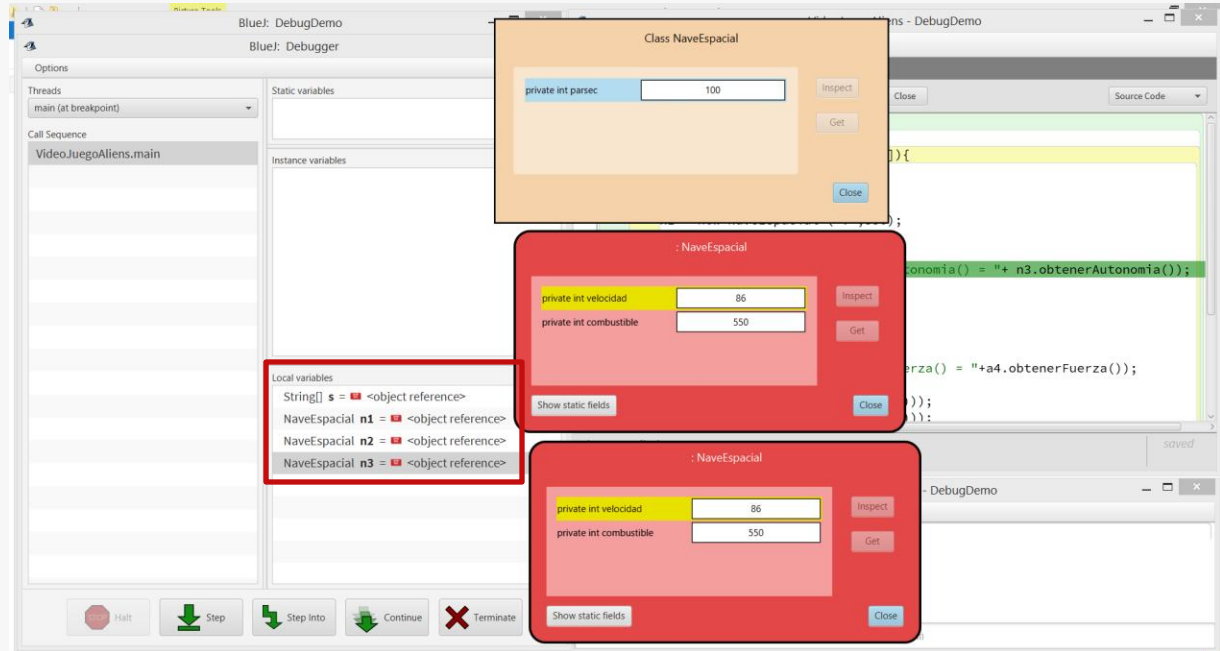


Iniciando el debugger en BlueJ

3. La vista de Debugger de BlueJ muestra información adicional de depuración en la sesión de depuración actual.

Los objetos en las Variables, pueden accederse para examinar sus atributos haciendo doble click en los mismos.

También, a partir de los atributos de los objetos se puede acceder a otros objetos referenciados y a los atributos de clase ([Show static fields](#))

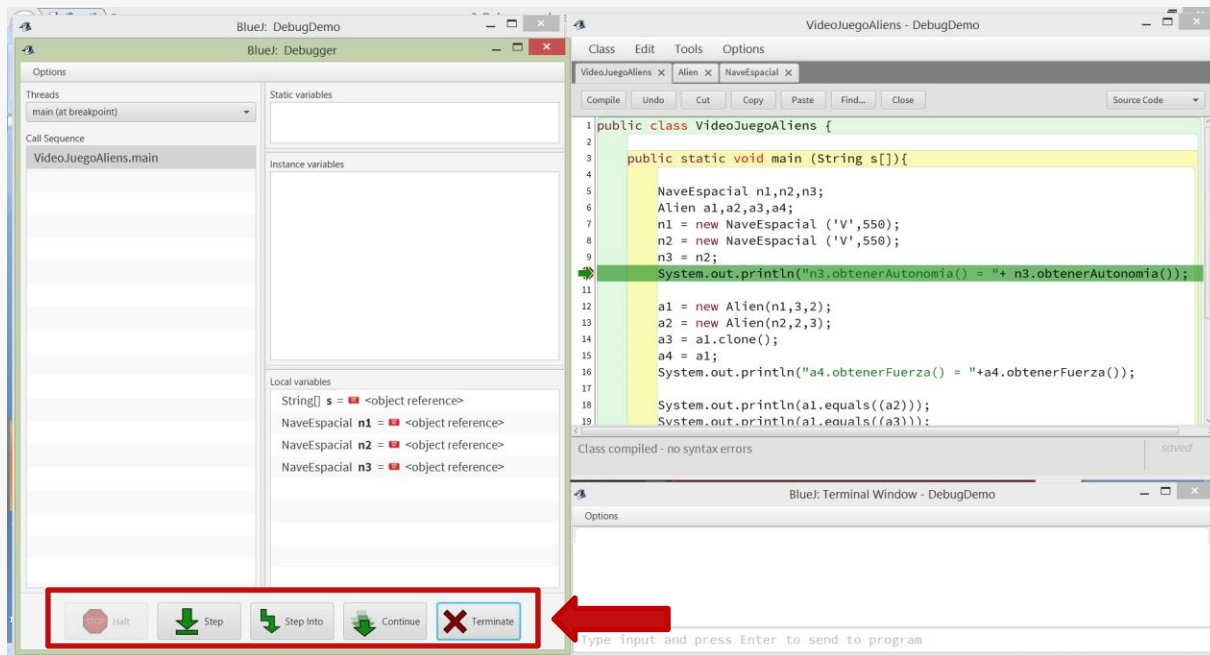


Controlando la ejecución

4. El control de la ejecución se realiza con la barra de botones de debug de la parte inferior.

Halt: Detiene la ejecución si la misma progresa sin interrupciones.

Step: Avanza la ejecución a la siguiente línea de código ejecutable sin ingresar en los métodos invocados.



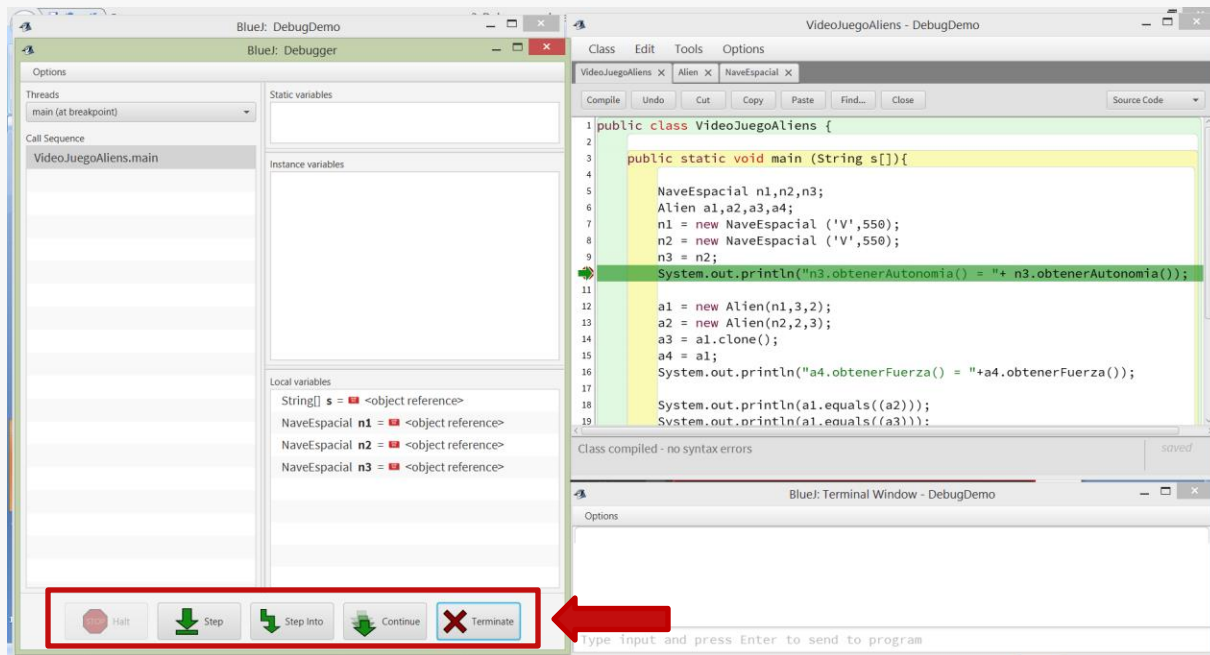
Controlando la ejecución

4. El control de la ejecución se realiza con la barra de botones de debug de la parte inferior.

Step Into: Ingresa al siguiente método invocado (según el orden de evaluación) y detiene la ejecución en la primera línea ejecutable del mismo.

Continue: Continúa la ejecución hasta alcanzar un breakpoint o terminar.

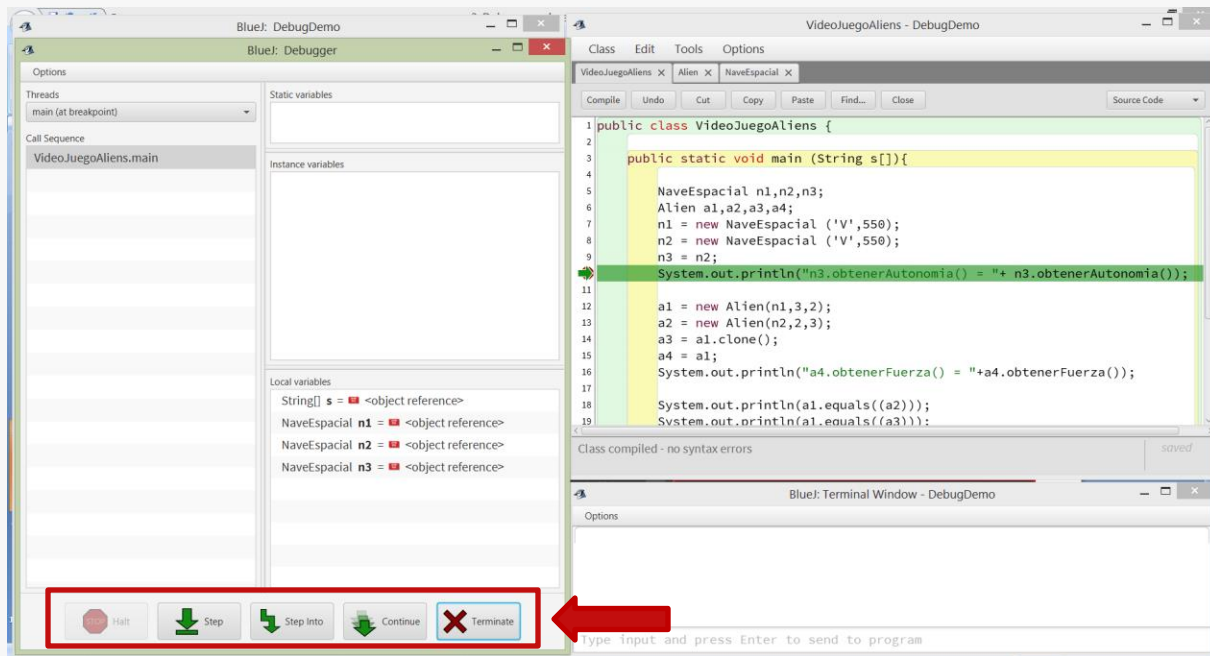
Terminate: Finaliza la sesión de debug actual.



Controlando la ejecución

4. El control de la ejecución se realiza con la barra de botones de debug de la parte inferior.

La depuración es un proceso hacia adelante. No tengo forma de retroceder la ejecución a un estado anterior. Si "me pasé" del punto de ejecución a analizar, debo finalizar la sesión con **Terminate** para recomenzarla nuevamente.



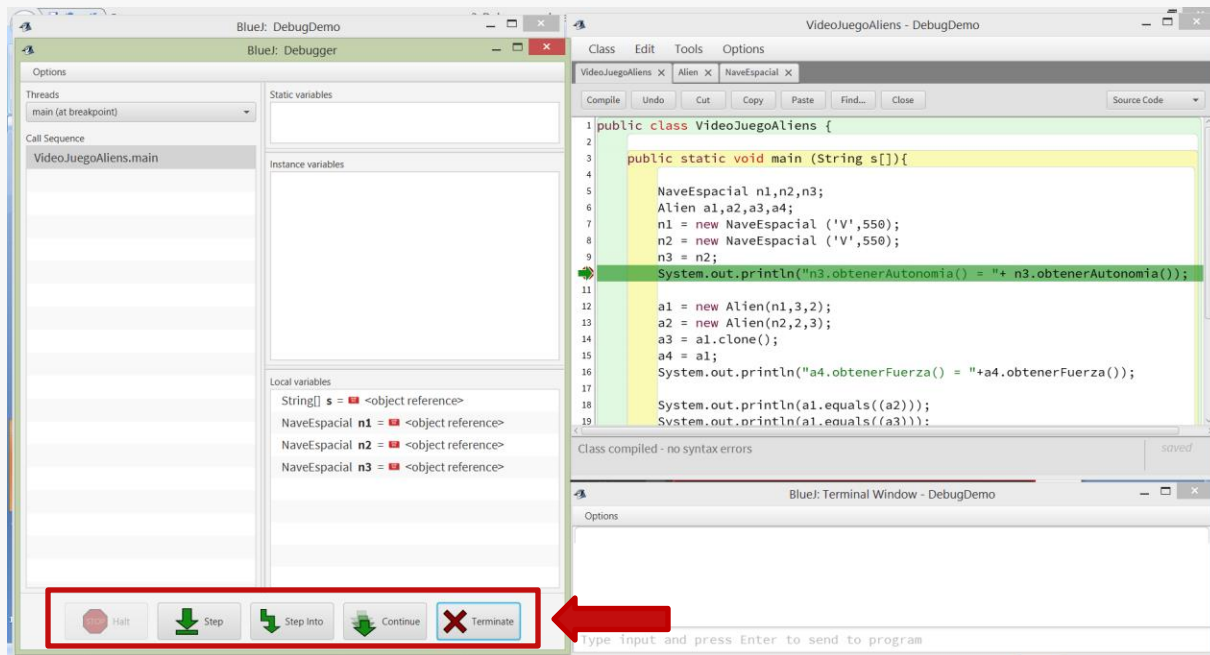
- • •
- • •
- • •
- • •
- • •

Controlando la ejecución

4. El control de la ejecución se realiza con la barra de botones de debug de la parte inferior.

La ejecución paso a paso (Step, o Step Into) nos permite examinar qué camino se toma en los condicionales, evaluar cómo iteran los loops, y seguir de cerca el flujo de ejecución.

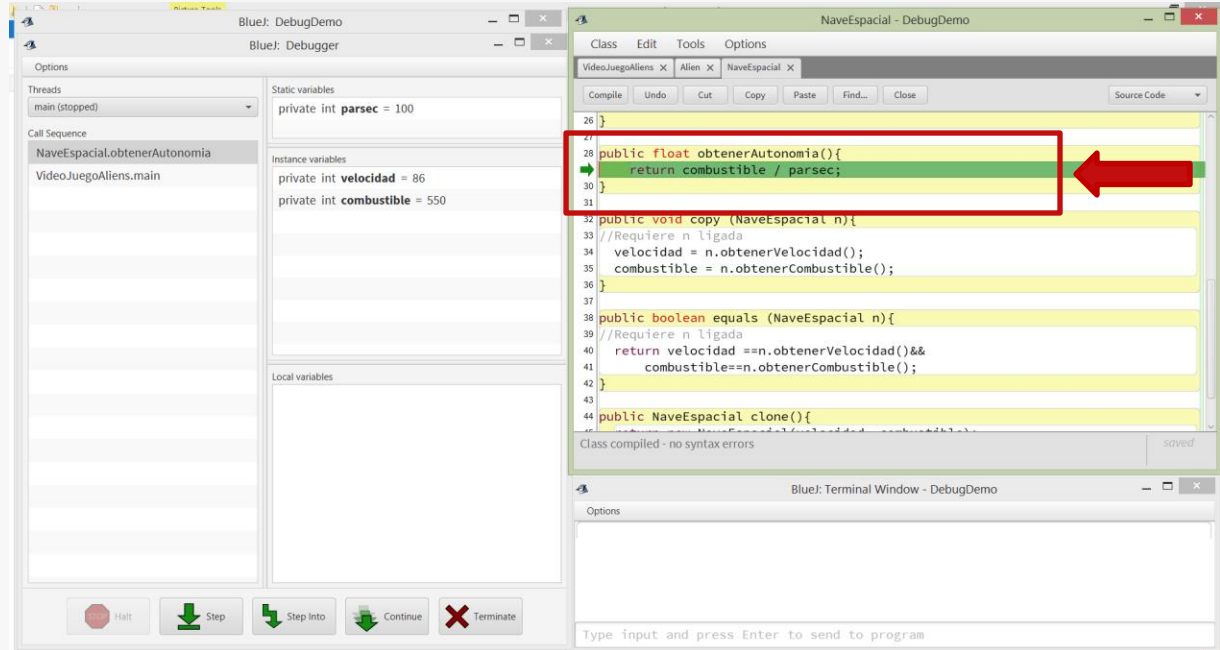
Siempre que se alcance un Breakpoint, la ejecución se interrumpe (aún durante Step o Continue).



Controlando la ejecución

4. El control de la ejecución se realiza con la barra de botones de debug de la parte inferior.

Al hacer **Step into**, el contexto cambia: pasamos a observar la ejecución del método `obtenerAutonomia()` en respuesta al mensaje enviado al objeto ligado a `n3`. La ejecución se encuentra detenida en la línea 29 de `NaveEspacial.java`, a punto de evaluar la división. Podemos ver los valores en el panel de variables para ayudarnos en esa evaluación ($550/100$).

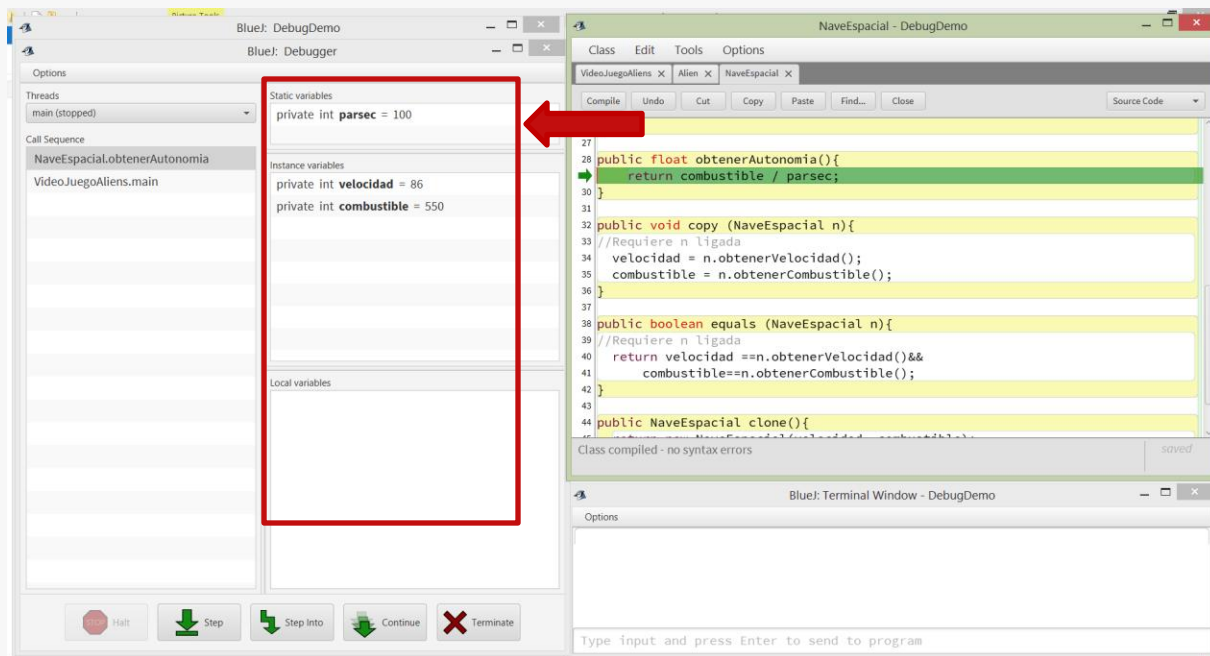


Controlando la ejecución

4. El control de la ejecución se realiza con la barra de botones de debug de la parte inferior.

Las variables y el ambiente **Local**, ahora son relativos al método `obtenerAutonomia()`.

Dado que en este caso no hay parámetros formales ni variables locales, sólo disponemos de los atributos de clase e instancia del objeto.

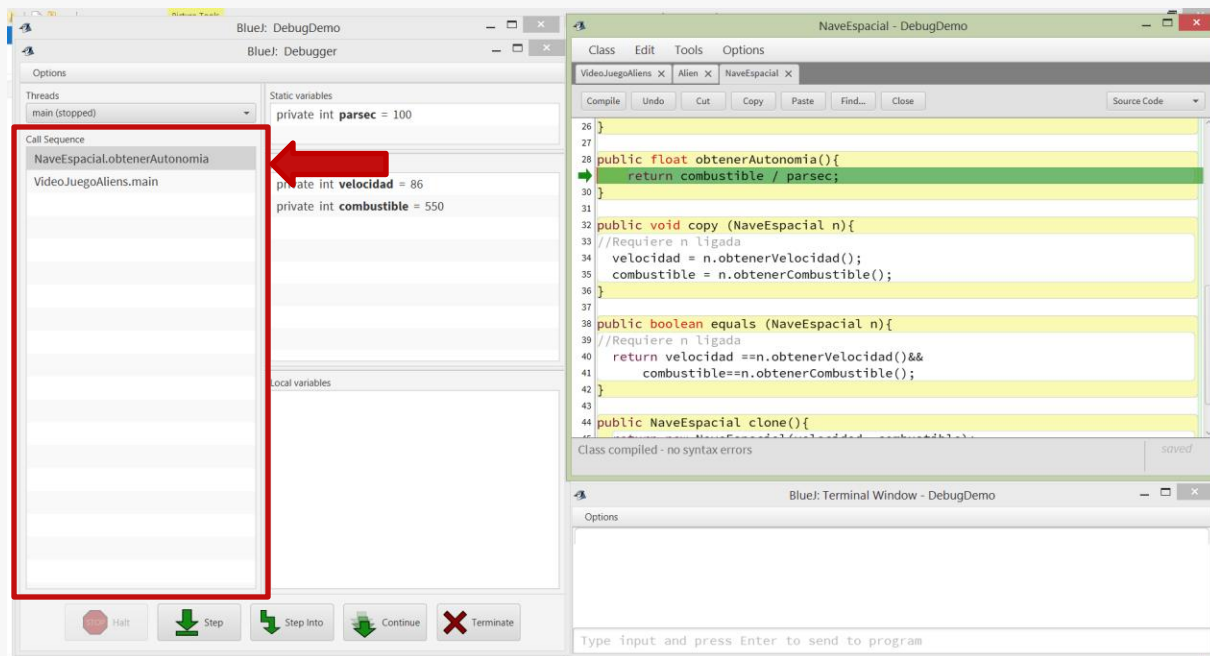


Controlando la ejecución

4. El control de la ejecución se realiza con la barra de botones de debug de la parte inferior.

Lo interesante ocurre en Call Sequence: ahí se pueden observar todos los métodos que están en ejecución al momento de la detención:

- main(...) de la clase VideoJuegoAliens, que invocó a
- obtenerAutonomia() de la clase NaveEspacial, que está detenido en la línea 29

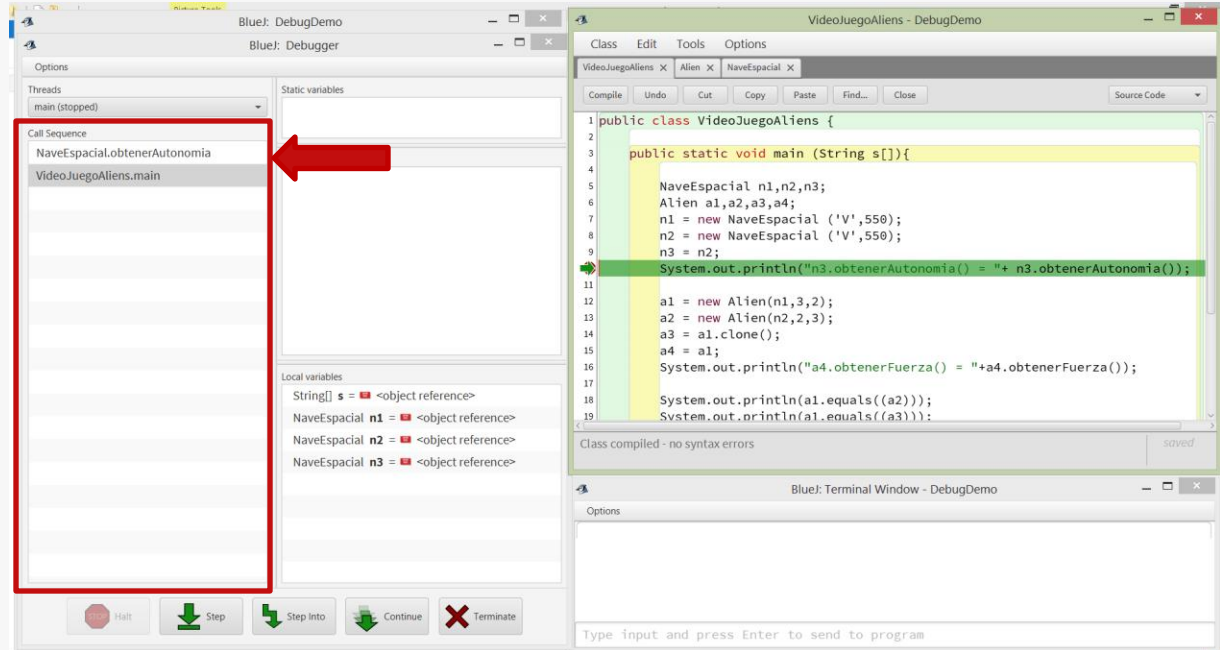


Controlando la ejecución

4. El control de la ejecución se realiza con la barra de botones de debug de la parte inferior.

Call Sequence nos brinda el contexto de ejecución de un método. Estamos analizando obtenerAutonomia() pero en el contexto en el que fue invocado desde main línea 10.

Haciendo click en cada línea de Call Sequence, podemos alternar el contexto y examinar en qué estado se encuentran los métodos invocadores/clientes.

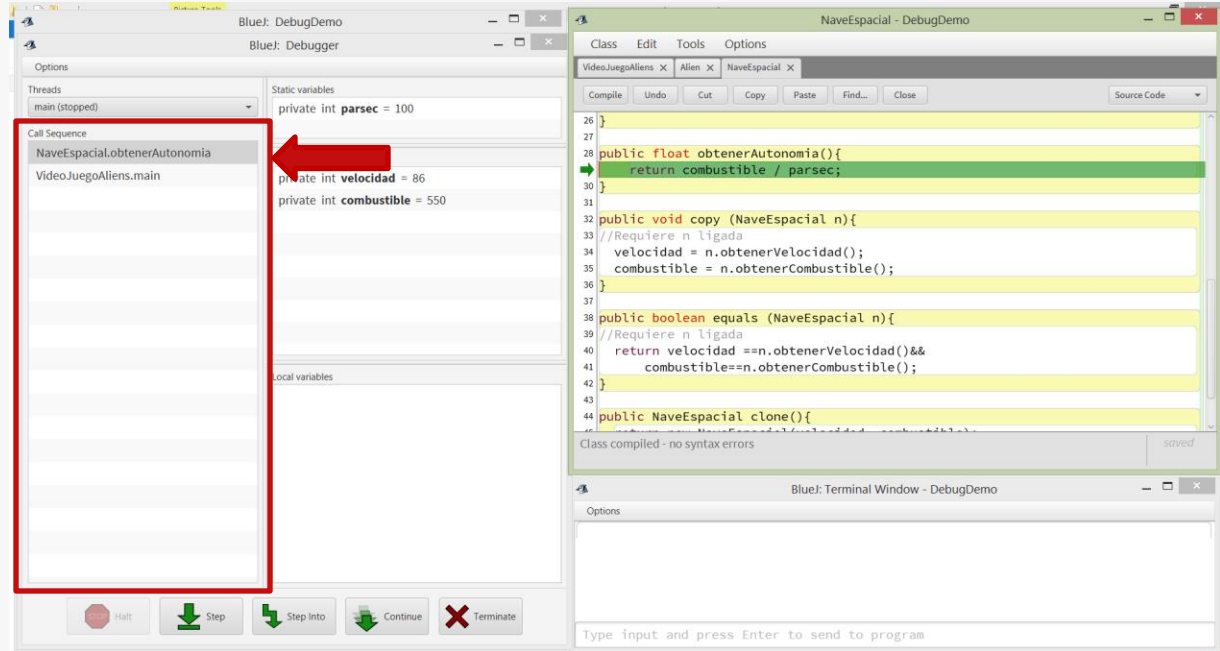


Controlando la ejecución

4. El control de la ejecución se realiza con la barra de botones de debug de la parte inferior.

Call Sequence es fundamental para establecer el contexto de ejecución de un método (quiénes lo invocaron y en qué estado).

Se utiliza mucho también en el reporte de errores de Java para dar contexto del error/excepción, mediante la denominada traza de pila (de llamadas) o Stack Trace.

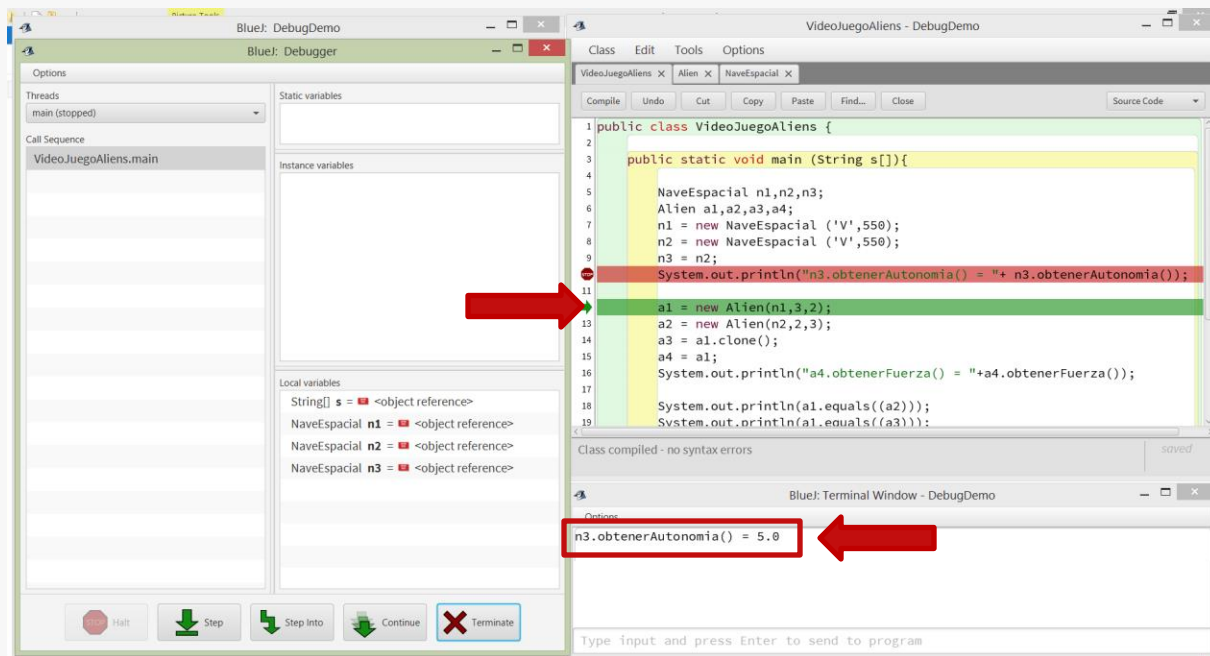


Controlando la ejecución

4. El control de la ejecución se realiza con la barra de botones de debug de la parte inferior.

Al continuar la ejecución un par de pasos, la misma retorna a main y completa la ejecución de la línea 10 mostrando por consola la autonomía calculada.

Si no somos cuidadosos acerca de a qué métodos entramos, podríamos terminar examinando métodos de Java como por ej. println, que no necesitamos ver.



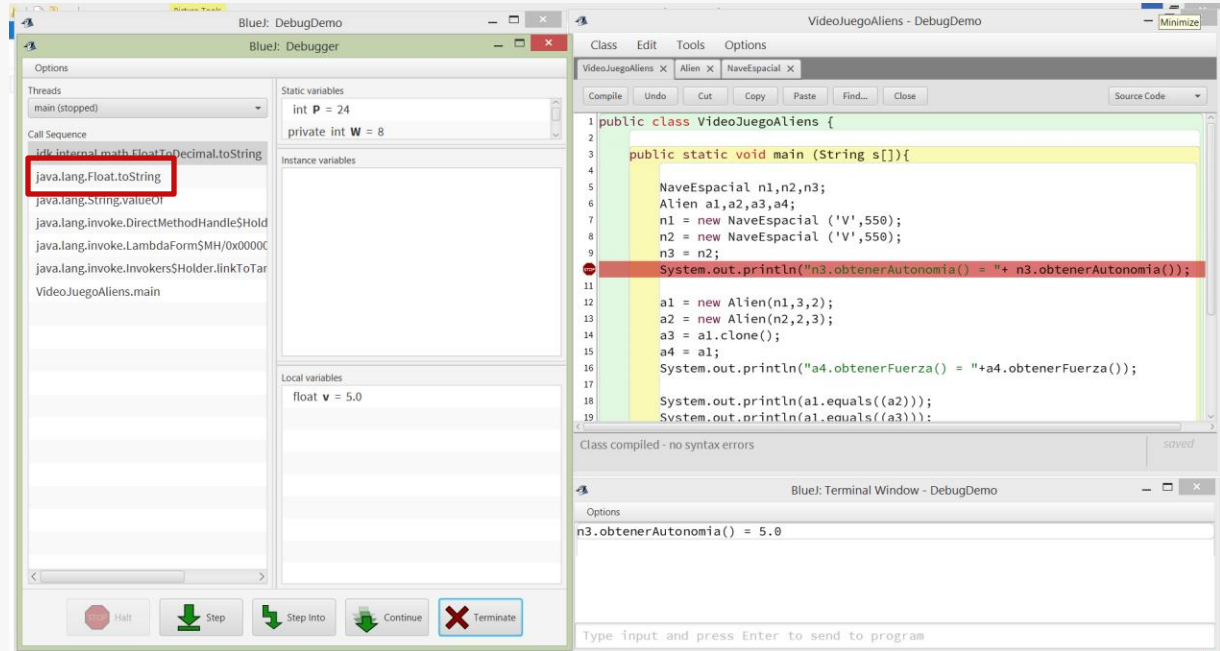
Controlando la ejecución

4. El control de la ejecución se realiza con la barra de botones de debug de la parte inferior.

Cosas que no necesitamos ver:

El código de conversión de float a String (como paso previo a concatenar Strings en línea 10 de main)

Dado que no se dispone del código fuente, el Debugger de BlueJ no nos puede mostrar en qué línea se encuentra la ejecución.





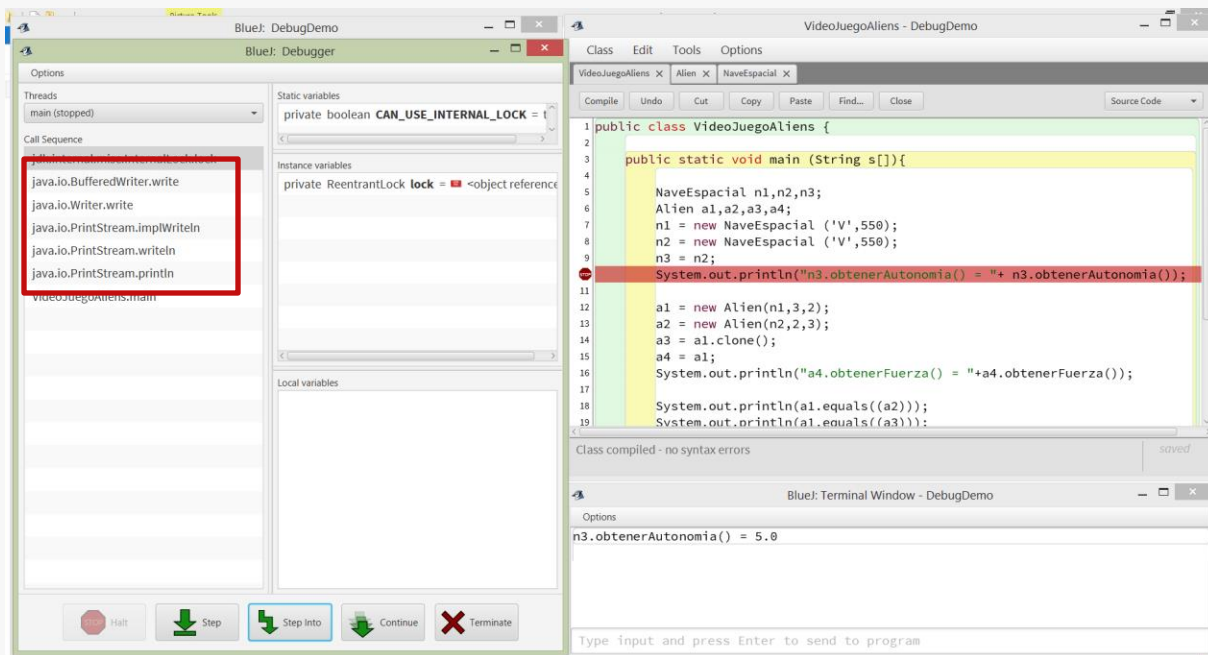
Controlando la ejecución

4. El control de la ejecución se realiza con la barra de botones de debug de la parte inferior.

Cosas que no necesitamos ver:

El código del println
invocado en línea 10 de
main.

El Debugger es capaz de
mostrarnos todo esto!





Controlando la ejecución

4. El control de la ejecución se realiza con la barra de botones de debug de la parte inferior.

¿Qué hacer si por error ingresamos en código que no deseamos ver?

- Analizar el contexto (Call Sequence) y continuar la ejecución tratando de salir de ahí (Step)
- Establecer un Breakpoint temporal en un punto posterior a la invocación del código al que se ingresó, para continuar (Continue) la ejecución hasta el mismo
- Simplemente, reiniciar la sesión de debugging y repetir los pasos para volver al contexto de ejecución previo a la invocación no deseada

La recomendación es utilizar Step Into sólo en casos muy específicos, ya que es preferible colocar Breakpoints en el código a invocar que se desea analizar.



Qué dejamos afuera en esta presentación

- El debugger de BlueJ es muy simple y básico, pero suficiente para poder depurar nuestros programas. Otras cosas que nos permiten hacer debuggers de otras herramientas son:
 - Establecer breakpoints condicionales
 - Establecer breakpoints basados en el disparo de excepciones y condiciones de error
 - Evaluar expresiones arbitrarias en el contexto de ejecución (watch)
 - Modificar el contenido de atributos y variables y continuar debuggeando
 - Aplicar técnicas avanzadas de debugging basadas en testing unitario como la sustitución de objetos (dummies, stubs, mocks, etc.) que escapan al alcance de la materia y serán abordados en materias más avanzadas del plan de estudios.





Consideraciones y recomendaciones finales

- El debugger nos permite examinar en detalle la ejecución de nuestro código. Esto es muy útil para entender mejor cómo funciona Java, cómo implementa los conceptos de Orientación a Objetos que estudiamos en la materia, y también para encontrar errores.
- Una vez hallado el lugar del/los error/es, es recomendable:
 - **finalizar la sesión de debug,**
 - **introducir las correcciones necesarias en el código y**
 - **recién después volver a debuggear para confirmar** que los errores fueron corregidos (y no se introdujeron nuevos).





Consideraciones y recomendaciones finales

- El debugger es una herramienta fundamental para el programador. Familiarícese con el mismo y aprendan a utilizarlo ya que les va a ahorrar muchísimo tiempo y esfuerzo de desarrollo y depuración.
- La principal ventaja del uso del Debugger es evitar ensuciar la lógica principal de nuestro código con instrucciones añadidas con el sólo propósito de conocer el estado de ejecución del mismo. El debugger nos permite ver el contenido de cualquier elemento de nuestro programa y seguir de manera detallada su flujo de ejecución. Úsenlo y dejen de lado técnicas de depuración más sucias tales como llenar de prints el código o comentar secciones enteras para tener idea de lo que está pasando.





¿Preguntas?

